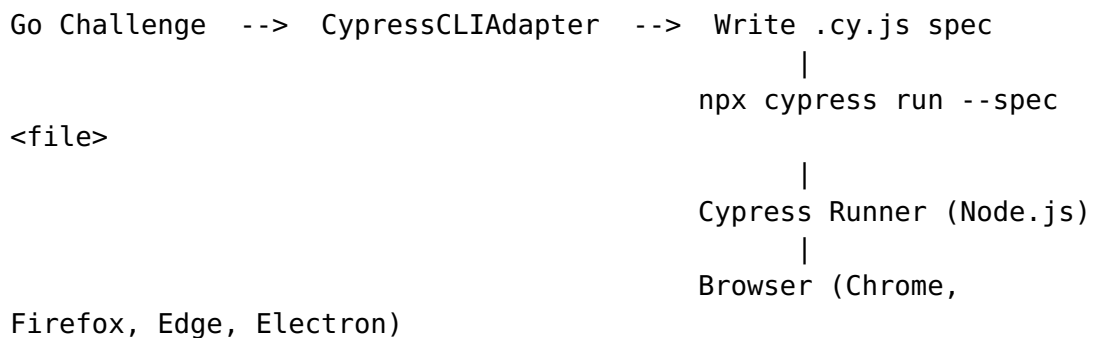


cypress-adapter

Cypress Adapter

The Cypress adapter implements BrowserAdapter by generating and executing Cypress spec files via the `npx cypress run` CLI. Each browser action produces a minimal `describe/it` spec, writes it to a temporary file, and invokes Cypress to run it.

Architecture



Unlike Selenium or Playwright, there is no persistent browser session. Each method call generates a self-contained spec file, runs it as an independent Cypress test, and parses the results from `stdout`. This approach trades performance for simplicity -- no long-lived connections or sessions to manage.

Spec Generation

The adapter generates Cypress specs dynamically. For example, `Click("#submit")` produces:

```
describe('action', () => {
  it('performs action', () => {
    cy.visit('http://localhost:3000');
    cy.get('#submit').click();
  });
});
```

Specs that need to return data (e.g., `GetText`, `IsVisible`) use `cy.task('log', ...)` to write JSON to stdout, which the adapter parses.

Prerequisites

1. **Node.js** (v16+) installed and in PATH
2. **Cypress** installed in the project: `npm install cypress --save-dev`
3. A valid Cypress configuration file (`cypress.config.js` or `cypress.config.ts`) in the project directory
4. For `cy.task('log', ...)` to work, the Cypress config must define a log task (or the adapter falls back to stdout parsing)

Constructor

```
adapter := userflow.NewCypressCLIAdapter("/path/to/project")
```

The argument is the project directory containing the Cypress configuration. The adapter defaults to Chrome in headless mode.

Configuration

Initialize

```
err := adapter.Initialize(ctx, userflow.BrowserConfig{
    BrowserType: "chrome", // "chrome", "firefox", "edge",
                  "electron"
    Headless:    true,
})
```

Creates a temporary directory for generated spec files. Browser type mapping:

BrowserConfig Value	Cypress Browser
chrome, chromium	chrome
firefox, gecko	firefox
edge, msedge	edge
electron	electron

API Reference

Navigate

```
err := adapter.Navigate(ctx, "https://example.com")
```

Generates a spec with `cy.visit(url)`. The URL is stored as `baseUrl` for subsequent specs that need to revisit the page.

Click, Fill, SelectOption

```
err := adapter.Click(ctx, "#submit-button")
err = adapter.Fill(ctx, "#email", "user@example.com")
err = adapter.SelectOption(ctx, "#country", "US")
```

- `Fill` uses `cy.get(selector).clear().type(value)`
- `SelectOption` uses `cy.get(selector).select(value)` (native Cypress dropdown support)

IsVisible

```
visible, err := adapter.IsVisible(ctx, ".alert-success")
```

Generates a spec that checks `$body.find(selector).is(':visible')` via jQuery (available in Cypress), logs the result via `cy.task`, and the adapter parses `"visible":true` from stdout.

WaitForSelector

```
err := adapter.WaitForSelector(ctx, ".dashboard", 10*time.Second)
```

Uses `cy.get(selector, { timeout: ms }).should('exist')` with the Cypress built-in retry mechanism.

GetText, GetAttribute

```
text, err := adapter.GetText(ctx, "h1.title")
href, err := adapter.GetAttribute(ctx, "a.link", "href")
```

Both generate specs that use `cy.invoke('text')` or `cy.invoke('attr', name)` and log the result as JSON via `cy.task('log', ...)`.

Screenshot

```
png, err := adapter.Screenshot(ctx)
```

Generates a spec with `cy.screenshot(name)`. Cypress saves the screenshot to `cypress/screenshots/`. The adapter then reads the PNG file from disk using glob pattern matching.

EvaluateJS

```
result, err := adapter.EvaluateJS(ctx, "return document.title")
```

The script is base64-encoded and injected into a spec that decodes and executes it via `cy.window().then(win => { ... })`. Results are logged via `cy.task`.

NetworkIntercept

```
err := adapter.NetworkIntercept(ctx, "**/api/**", handler)  
// Returns nil; handler will NOT fire.
```

Not supported in the CLI adapter. Cypress does support `cy.intercept()` natively, but the Go callback cannot be wired to a generated spec. For network interception, write custom Cypress specs directly.

Close, Available

```
err := adapter.Close(ctx)           // Removes temp directory  
ok := adapter.Available(ctx)       // Runs "npx cypress --version"
```

Limitations

Aspect	Detail
Performance	Each method spawns a full Cypress process. Significantly slower than Selenium or Playwright
Cross-browser	Chrome-family and Firefox. No Safari or mobile. Less cross-browser coverage than Selenium
Network interception	Not available through the CLI adapter. Must use <code>cy.intercept()</code> in hand-written specs
State persistence	No persistent session. Each spec starts fresh. The baseURL is revisited per spec
Data extraction	Relies on stdout parsing via <code>cy.task('log', ...)</code> . Fragile if Cypress output format changes
Parallelism	Cypress does not support running multiple specs concurrently in a single process

When to Use

- Your project already uses Cypress and you want to integrate existing specs into the challenge framework
- You need Cypress-specific features (automatic retries, time travel debugging, `cy.intercept()`)
- Speed is not critical and you prefer Cypress's developer experience

Integration with Challenge Templates

```
adapter := userflow.NewCypressCLIAdapter("/app/frontend")

challenge := userflow.NewBrowserFlowChallenge(
    "CH-CYPRESS-001",
    "Cypress Registration Flow",
    "Verify user registration via Cypress",
    nil,
    adapter,
    userflow.BrowserFlow{
        Name:      "cypress-register",
        StartURL:   "http://localhost:3000/register",
        Config: userflow.BrowserConfig{
            BrowserType: "chrome",
            Headless:    true,
        },
        Steps: []userflow.BrowserStep{
            {Name: "fill-name", Action: "fill", Selector: "#name", Value: "Test User"},
            {Name: "fill-email", Action: "fill", Selector: "#email", Value: "test@test.com"},
            {Name: "submit", Action: "click", Selector: "button[type=submit]"},
            {Name: "verify", Action: "assert_visible", Selector: ".success-message"},
        },
    },
)
```

Source Files

- Interface: `pkg/userflow/adapter_browser.go`
- Implementation: `pkg/userflow/cypress_adapter.go`